

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE



ROBOTIQUE, 2025-26

Master thesis

DASH-01 :
RL Framework to Develop a Bipedal
Humanoid Robot Aimed at Breaking the
Speed Record

REHASSIST
13.04.2025 - 04.09.2025

Author:

Nathann MORAND
Robotics M.Sc.

Supervisors:

Dr. Mohamed BOURI

Abstract

Table of Contents

Abstract	2
1 Introduction	5
1.1 Problem statement	5
1.2 Objectives of the Project	5
1.3 Methodology Overview	5
1.4 Contributions	5
2 Background and State of the Art	6
2.1 Dynamic biped locomotion	6
2.2 Control approaches	6
2.3 RL for locomotion	6
2.4 Existing robot platforms	6
3 Development Methodology	7
3.1 Overall methodology	7
3.2 Hardware selection	7
3.2.1 Motor	7
3.2.2 Battery pack	7
3.2.3 Compute	7
3.2.4 Sensors	8
3.3 Robot modelling	8
3.4 Simulation	8
3.5 Toy problem / RL Warmup	8
3.6 RL and validation	8
3.7 deployment	9
3.7.1 linux and RT	9
3.7.2 CAN	9
3.7.3 motor controller	9
3.7.4 hardware validation	9
3.8 Hardware experiments	9
4 Results	10
4.1 Hardware selection	10
4.2 RL experiments	10
5 Discussion	11
5.1 Successes	11
5.2 Failures	11
5.3 Lessons learned	11
6 Conclusion and Future Work	12
6.1 Summary of Findings	12
6.2 Challenges and Limitations	12
6.3 Suggestions for Future Research	12
7 Appendix	13
7.1 logbook	13

References	14
------------	----

1 Introduction

This thesis investigates the practical challenges of applying existing RL methods to a custom high-speed biped robot and provides a reproducible development methodology covering robot modelling, simulation, control integration, and experimental validation.

1.1 Problem statement

1.2 Objectives of the Project

1.3 Methodology Overview

What tool / technic, ... did I use / applied my strategy to solve the problem

1.4 Contributions

What I did, recap of my original work.

What I did : actuator mapping database with metric for figure of merit URDF modelling from scratch linux, CAN, cubemars setup Battery pack design and performance RL toy problem -> Spinoff as TP ? RL for running robot, incremental approach and test -> torque control -> no convergence, too slow with RT linux -> Position control -> converged before update mass and inertia, not converging anymore... -> gait generator approach, fourrier coefficient, simplified problem with reduced DOF, hypothesis that the abduction alone can stay oin position control and be enough to act as reflex system to keep robot stable. running with the based attach on a threadmill. -> attempt to train for fully unthethered operation (not done yet)

things to develop

map motor power density how to URDF from CAD correctly. (manual process)

state of the art running robot mechanical (model (SLIP, oscillator,) , timming estimation, ... state of the art RL and bipedal + sim2real sensor choice BOM development electrical system (and control issue + timming and jitter) toy problem to test the system and setup manual for student TP real implementation and training real deployment and challenge

2 Background and State of the Art

Hype in running robot, recent achievement, ...

2.1 Dynamic biped locomotion

the models that exist, why its hard, how to stabilize

2.2 Control approaches

classical vs RL, why RL is interesting

2.3 RL for locomotion

PPO action spaces reward design sim-to-real and all other things I tried / read (fourier, torque vs position, ...)

2.4 Existing robot platforms

im not sure I care much about it. done already by max anyway...

3 Development Methodology

3.1 Overall methodology

hybrid start with robot partially designed, Robot modelling and help with hardware selection for non pure-mechanical part URDF export Robot workspace Validation and calibration Robot dynamical validation against inertia matrix and URDF Simulation and fidelity implementation Toy problem for RL complete loop with one actuator, one linux on BBB, CAN bus, ... to gain experience, RL training and validation (different approach tried) deployment consideration (linux and RT/semi RT kernel), CAN bus bandwidth and refresh speed, motor control mode, hardware validation, back driving and play back,

3.2 Hardware selection

3.2.1 Motor

3.2.2 Battery pack

having select the motor, we knew the current that would be needed to drive them. (

We need to optimise for power density as the robot run are quite short (after all, it took usain bolt less than 10 second to run 100m, I hope our robot can do it in less than 30) Its not a marathon running robot but a pure speed sprinter one.

maybe highlight mismatch between commercial offering and theoretical performance by doing a map like for the motors.

Thus I went one to review cell and chemistry for power density. And while LFP and LTO along with Lipo should be more power dense, some challenge in sourcing theses cell made them an unviable option. Commercial LTO cell where found on ALiexpress but decided as not worth the risk. I could not find any reputable / specialized suppliers that would ship cells. Nkon is the the supplier I have used in the past for battery need and from there I looked at the most power dense battery cell.

The EVE INR21700-50PL 5000mAh - 125A (nieuwe versie) where selected

3.2.3 Compute

Most of the lab exoskeleton run on beaglebone black While theses support CAN natively, and run linux, the SSH connection was VERY slow and thus was rejected. The main selling point was the real time coprocessor (understand a beefy MCU that run alongside the main CPU for time critical application) but this require custom code and the performance benefit are still to demonstrate.

A raspberry pi 2 b+ that was laying around was used instead as the author had much more experience with theses. It did require to buy a CAN hat over SPI. another hat with an accelerometer was bought (raspberry pi sense hat) for the ease of integration.

mention here or later the need to check the mapping when stacking the two hats.

Need to document the test for loop frequency and jitter but main result is that we can have a can loop at 200 Hz but and that the raspberry pi is MUCH more powerful than the beagle bone. which is very convenient for development (we host a wifi hotspot so we can connect to the robot, we host a web interface to control the robot to simplify the manipulation)

3.2.4 Sensors

sense hat for base IMU, contact sens but never tried. literature suggested that impact timing and direction can improve RL convergence.

3.3 Robot modelling

Re assembled the robot in autodesk inventor (for fluency reason) and exported as STEP each limb with the origin centered at the joints.

Assembled the robot in fusion and used plugin to export URDF

Manually exported inertia matrix from solidworks, provided by mechanical engineer but matrixes appear in wrong orientation and after multiple attempt, the URDF export did not yielded a usable URDF.

CAD matrix, validate accuracy and orientation with reference frame. Made tool to discover plausible inertia matrix from robot and validate against known one (since safe workspace is known, we get the robot to move with fixed base, leg in the air)

3.4 Simulation

mujoco, URDF limitation with loop, GPU vs CPU, ...

3.5 Toy problem / RL Warmup

3.6 RL and validation

what I tried (torque, position, pattern generator, ...) does it work ? sim2sim ? sim2real ? ...

need to rework software to streamline these experiments.

should work for anything between the pendulum and the running robot (and explain how to adapt to new case)

we want to be able to test : -i spring + damper + friction -i PID control -i torque control -i Position control -i pattern generator tuning (number of weight) and this independently for each joint

for each segment we want to be able to arbitrarily restrict DOF (the pendulum base is fully restricted while for the running robot we can try to restrict or not DOF.

we want a training script that allow to configure each training with the number of epoch (given that there is anything to train) and output a plot of the training epoch + csv

we want a reward policy definition script where we can easily test things -i need to be defined per experiment

we want a visualisation script to test the performance of our policy / system, it should give us mp4 video of a run but also allow to give command if the policy should need it (like if I want to manually steer my robot)

-i how do we configure the network architecture for RL ? with the user input too ? -i need to be defined per experiment

-i how do we define curriculum learning ? -i need to be defined per experiment

-i how do I batch experiment and monitor progress ?

-¿ how do we support both CPU and GPU and optimise parameters for maximum training speed (number of env, ...)

-¿ how do we integrate the robot workspace safety approach ? and use that as a kind of sim2sim ?

what strategy for the running robot ? like it might be better to train versatile simple policy than specific one (teach it to run only instead of run for 100m then stop)

3.7 deployment

3.7.1 linux and RT

3.7.2 CAN

3.7.3 motor controller

3.7.4 hardware validation

webUI with simple path, safe limits, ...

3.8 Hardware experiments

4 Results

4.1 Hardware selection

show motor, battery pack, ...

4.2 RL experiments

show all the practical result we learned.

5 Discussion

5.1 Successes

What worked? pipeline modelling standing gait formulation

5.2 Failures

What did not work? direct torque RL fully autonomous running

5.3 Lessons learned

Simulation fidelity matters. Action-space design matters. Hardware integration dominates development time. Progressive validation reduces debugging complexity.

6 Conclusion and Future Work

6.1 Summary of Findings

6.2 Challenges and Limitations

6.3 Suggestions for Future Research

7 Appendix

7.1 logbook

- 2025-02-20** Creation of this report, rough layout of the idea in chapter
- 2025-02-25** Longitudinal kinematic model, research about markov modelling and cycle
- 2025-02-26** dynamic simulation creation, implemented a basic simulation in pybullet with keyboard navigation and parametric surface + wheel rolling down
- 2025-02-27** refining efficiency to take into account whole cycle, reading paper
- 2025-02-28** Writing Idea about improving efficiency through identified parameters
- 2025-03-04** Adding graph about car occupancy and real world driving needs + reading paper
- 2025-03-06** Brainstorming kinematic, reading PhD thesis on narrow track
- 2025-03-09** URDF is a PAIN to implement to due lack of tool, trying to define a kinematic model in python
- 2025-03-11** reworking part of chapter 2
- 2025-03-17** reading paper about cars and bike and limiting factor to adoption
- 2025-03-18** keep reading paper, also about narrow track vehicle
- 2025-03-20** trying to structure what I learned from the papers
- 2025-03-21** defining the know unknown about car adoption factor (human side)
- 2025-03-24** Clarifying the project goal
- 2025-03-27** draw figure, added citation
- 2025-03-28** Manual definition is an epic failure... Two way (createMultiBody -; Works but hard to read), Create body and add constraint don't work (rotation constraint crash the simulation.)
- 2025-04-09** Realized that createMultibody is more stable but has a "ghost" friction problem, Might as well use URDF then.
- 2025-04-10** learning URDF and how the origin is defined,
- 2025-05-25** redefined most variant as URDF and how to simulate them. Need spend time to implement basic controller like PID to show steady state stability and control margin
- 2025-05-28** Meeting, dry run presentation and shown current status.
- 2025-05-31** Report restructuring, writing
- 2025-06-02** Chapter 4, 5, 6 writing
- 2025-06-04** Getting simulation result, finishing first draft of report.
- 2025-06-08** Proof reading

References